

Orchestrating AI Workloads with Kubernetes

A declarative, resilient, and scalable foundation for running training jobs, batch ETL, real-time inference, and enabling CI/CD, autoscaling, and multi-tenant operations.



What Kubernetes Gives You



Self-healing

Automatically restarts failed pods, ensuring continuous service availability

Horizontal scaling

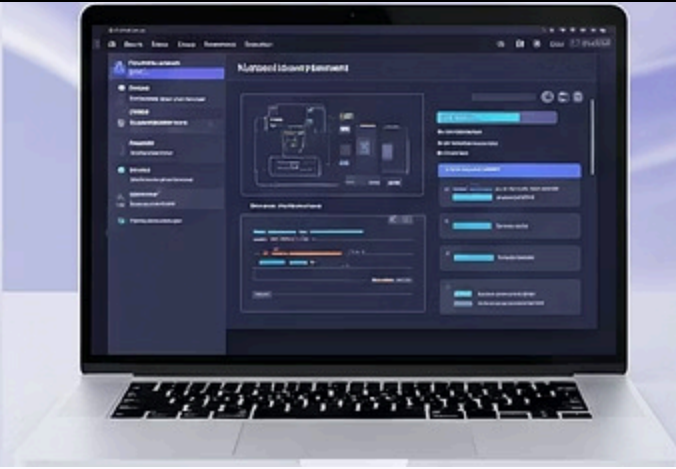
Dynamically adds replicas under load to maintain performance

Service discovery

Provides stable DNS names for pods, simplifying networking

Isolation & quotas

Enforces boundaries with Namespaces, RBAC, and ResourceRequests/Limits



Cluster Setup Options

Local Development

kind / minikube

- Perfect for development environments
- Runs on your local machine
- Easy setup and teardown

Managed Production

GKE / EKS / AKS

- Production-grade infrastructure
- Managed control plane
- Automatic upgrades

GPU Configuration

NVIDIA Device Plugin

- Enable GPU instance types
- Install NVIDIA device plugin
- Access GPU acceleration for ML workloads

Core Kubernetes Objects Refresher



Pod

The smallest deployable unit containing one or more containers that share storage and network resources



Deployment

Manages desired replica count and enables rolling updates with version control



Service

Provides a stable virtual IP address and DNS name for a set of pods



Ingress

Manages external HTTP routing to Services, often with TLS termination



ConfigMap/Secret

Stores configuration parameters and sensitive credentials separately from application code

Minimal Inference Deployment

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nlp-inference
  labels:
    app: nlp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nlp
  template:
    metadata:
      labels:
        app: nlp
    spec:
      containers:
        - name: api
          image: ghcr.io/acme/nlp:1.0.0
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: "500m"
              memory: "1Gi"
            limits:
              cpu: "1"
              memory: "2Gi"
---
apiVersion: v1
kind: Service
metadata:
  name: nlp-svc
spec:
  selector:
    app: nlp
  ports:
    - port: 80
  targetPort: 8080
  type: ClusterIP
```



This minimal deployment creates a scalable NLP inference service with appropriate resource allocation.

GPU Scheduling

```
spec:  
  template:  
    spec:  
      containers:  
        - name: triton  
          image: nvcr.io/nvidia/tritonserver:24.05-py3  
      resources:  
        limits:  
          nvidia.com/gpu: 1 # request a GPU
```

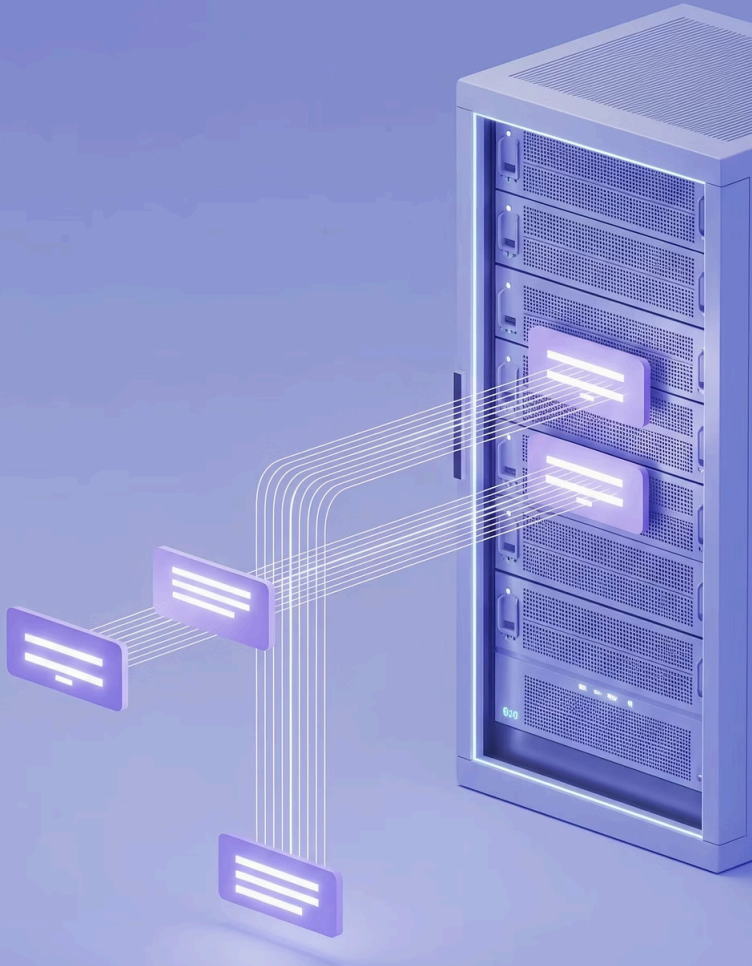
Key Requirements:

- Install **NVIDIA device plugin** on the cluster
- Ensure image CUDA/cuDNN versions match node drivers
- Use appropriate node selectors for GPU-specific nodes
- Consider fractional GPU allocation for better utilization



The NVIDIA device plugin enables Kubernetes to discover and schedule GPUs as a cluster resource, just like CPU and memory.

External Traffic with Ingress



```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nlp-ing
  annotations:
    kubernetes.io/ingress.class: "nginx"
spec:
  rules:
  - host: api.capstone.local
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: nlp-svc
            port:
              number: 80
```

For managed cloud environments, combine **Ingress + LoadBalancer** service types and add TLS certificates via cert-manager for secure HTTPS connections.

Config & Secrets

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: nlp-config
data:
  MODEL_NAME: "distilbert-qa"
```

```
apiVersion: v1
kind: Secret
metadata:
  name: nlp-secrets
type: Opaque
stringData:
  API_KEY: ""
```

```
# Mount into pod
env:
- name: MODEL_NAME
  valueFrom:
    configMapKeyRef:
      name: nlp-config
      key: MODEL_NAME
- name: API_KEY
  valueFrom:
    secretKeyRef:
      name: nlp-secrets
      key: API_KEY
```

Best Practices:

Separate concerns

Keep application code, configuration, and secrets independent for easier management

Use references

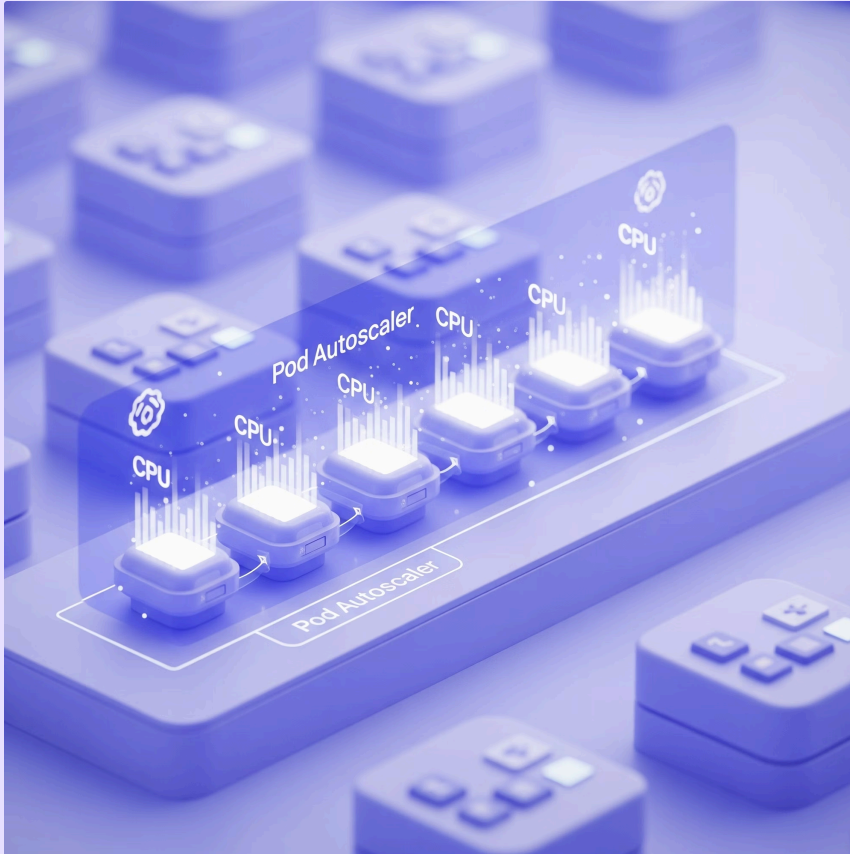
Reference ConfigMaps and Secrets instead of hardcoding values

External secret stores

Consider HashiCorp Vault or cloud KMS for production secrets



Autoscaling with HPA

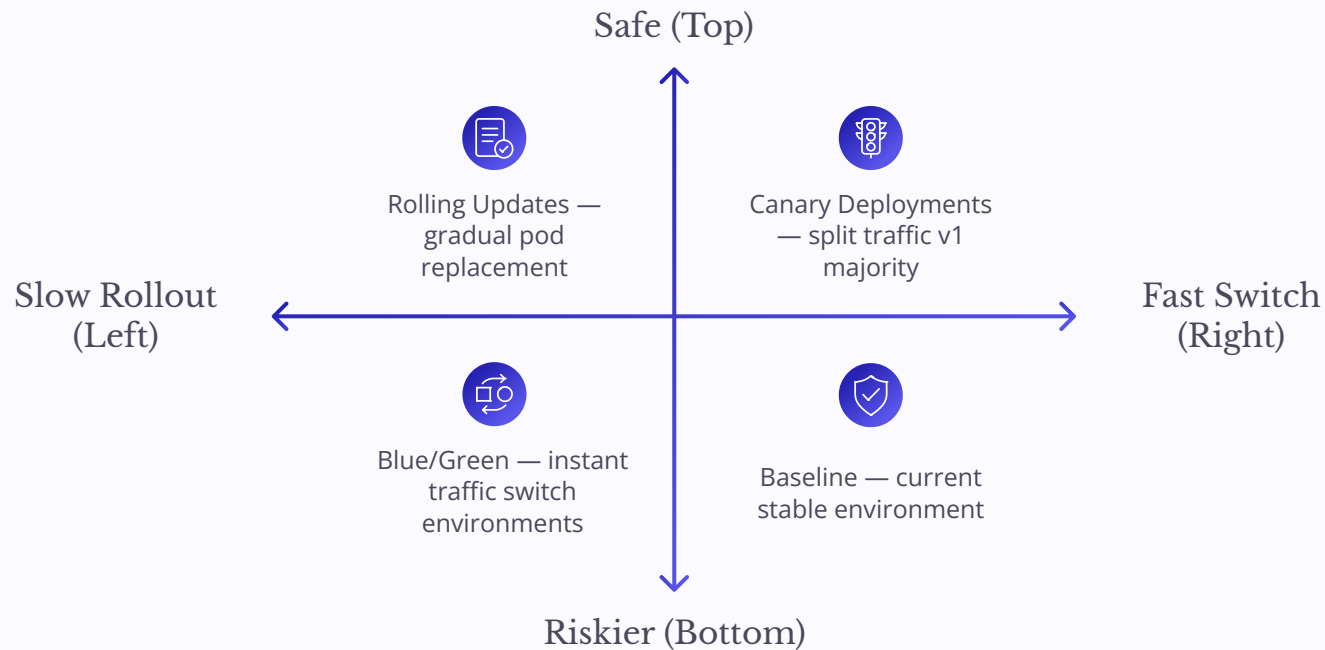


The Horizontal Pod Autoscaler automatically scales your deployment based on observed metrics, ensuring optimal resource utilization.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nlp-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nlp-inference
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70
```

- ❑ For GPU or latency-aware scaling, expose custom metrics such as requests/second or p95 latency using the Prometheus Adapter.

Canary & Blue/Green Releases



Rolling Updates (Default)

- Gradually replaces old pods with new ones
- Best for small, backward-compatible changes
- Simplest approach with minimal configuration

Canary Deployments

- Run v1 and small % v2 via two Deployments
- Use weighted Ingress or Service Mesh
- Allows real-world testing with limited exposure

Blue/Green Deployments

- Switch Service selector from blue to green atomically
- Enables instant rollback
- Requires double the resources temporarily

Batch Training with Jobs/CronJobs

One-time Training Job

```
apiVersion: batch/v1
kind: Job
metadata:
  name: train-run
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: trainer
        image: ghcr.io/acme/train:1.0.0
        args: ["--epochs", "3",
              "--out", "s3://bucket/checkpoints/"]
```

Scheduled Retraining

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-retrain
spec:
  schedule: "0 2 * * *" # 2 AM daily
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: Never
          containers:
          - name: retrain
            image: ghcr.io/acme/train:latest
```

Jobs execute once to completion, while CronJobs run on a schedule—perfect for periodic model retraining with fresh data.

Helm & Kustomize



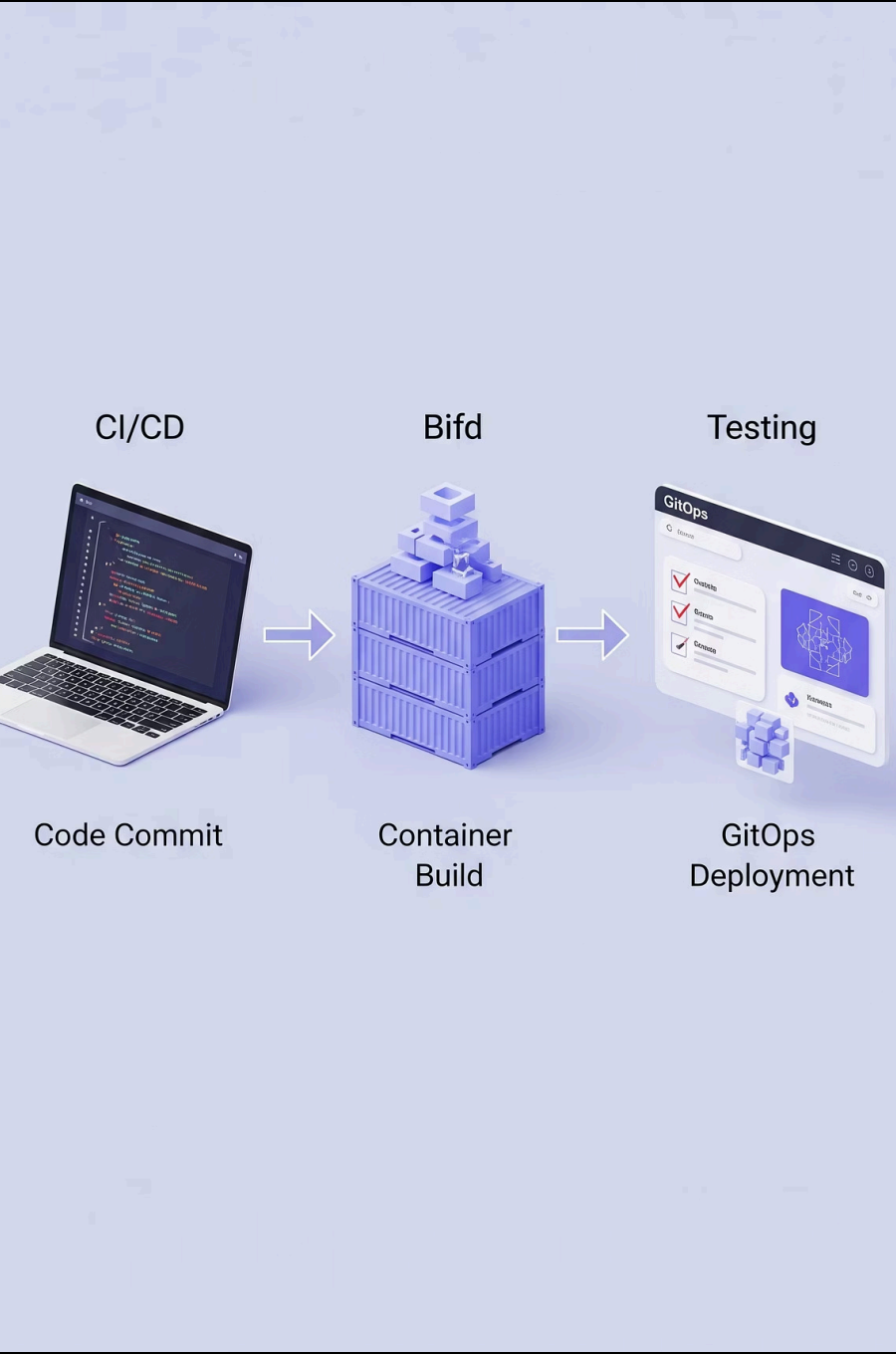
Helm

- Package manager for Kubernetes
- Parameterize charts with values.yaml
- Versioned releases with easy rollbacks
- Templating engine for complex deployments

Kustomize

- Native to kubectl (kubectl apply -k)
- Overlay bases for different environments
- No templates—just patches and transformers
- Composable with minimal duplication

Use one (or both) to avoid YAML copy-paste sprawl and improve maintainability.



CI/CD for Kubernetes



Code Push

- Developer commits code
- Triggers CI pipeline



Build & Validate

- Build Docker image
- Run unit tests
- Scan for vulnerabilities



Lint & Verify

- Lint Kubernetes manifests
- Validate with kubeval
- Policy checks with conftest



GitOps Deploy

- Update manifests in Git
- Argo CD/Flux reconciliation
- Pre-prod smoke tests

Observability Stack

Metrics

Prometheus + Grafana dashboards for CPU, GPU, QPS, and latency p95/p99 measurements

Logs

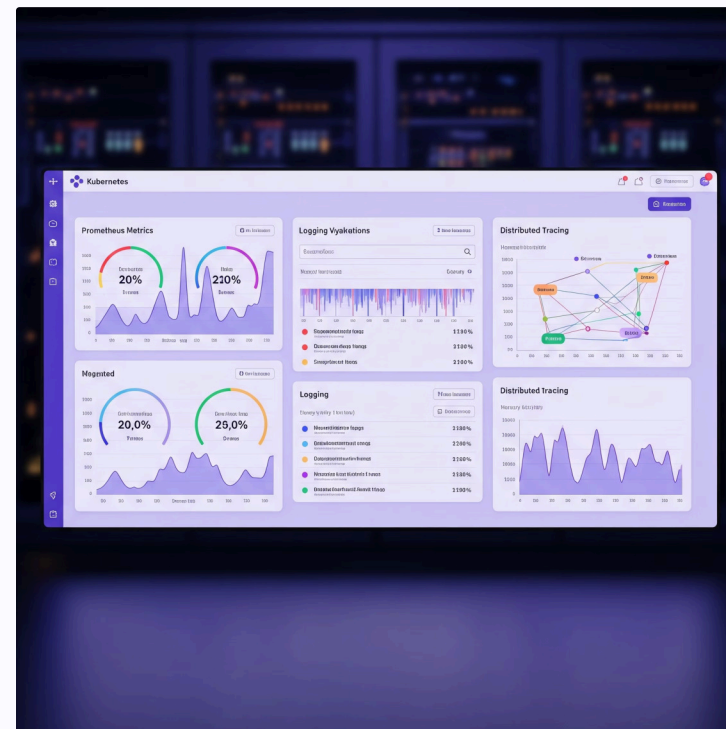
EFK/ELK Stack (Fluent Bit, Elasticsearch, Kibana) for centralized logging

Tracing

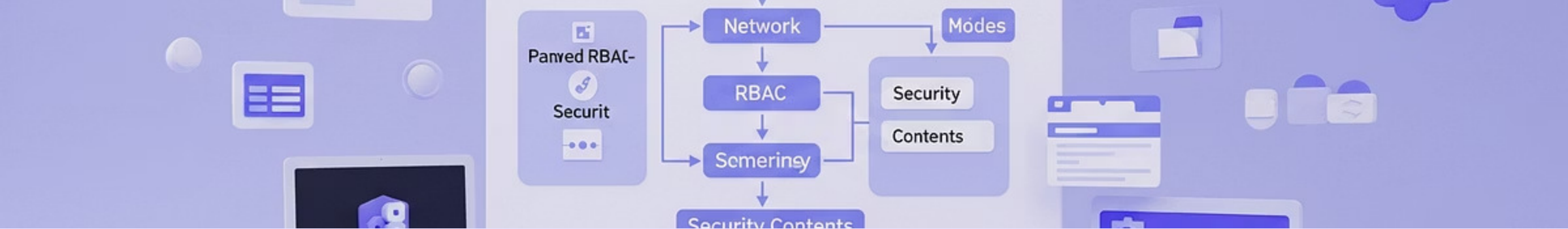
OpenTelemetry + Jaeger for distributed tracing across microservices

Health Monitoring

Readiness/liveness probes and DCGM for GPU health monitoring



A comprehensive observability stack provides insight into your AI workloads' performance, helping identify bottlenecks and troubleshoot issues quickly.



Security & Multi-Tenancy

Access Control

RBAC + Namespaces

- Role-Based Access Control
- Namespace isolation
- ResourceQuotas for fair sharing

Network Security

NetworkPolicies

- East-west traffic limitations
- Ingress/egress controls
- Zero-trust networking model

Secrets Management

External Vault

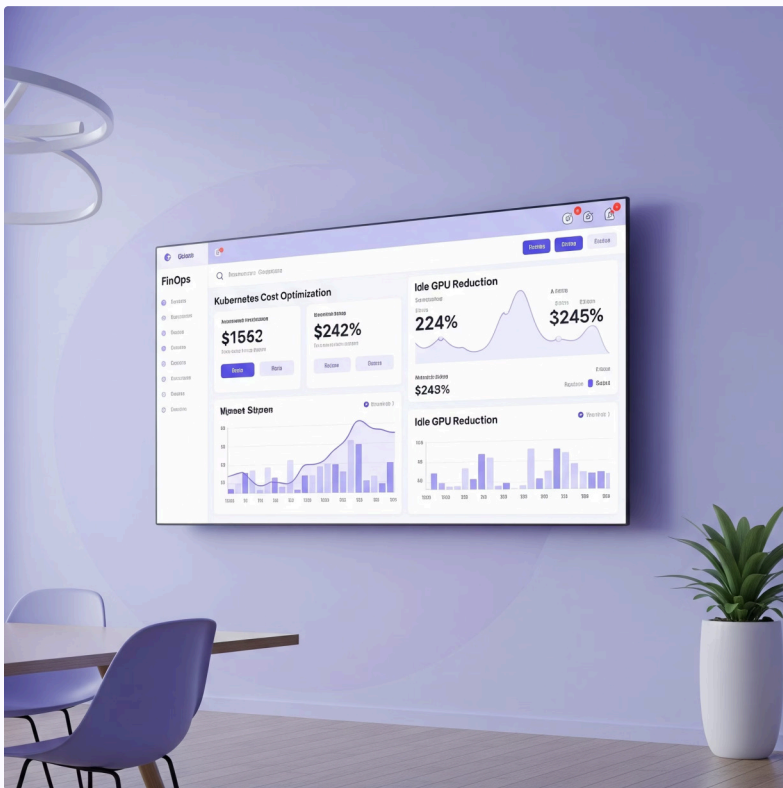
- KMS integration
- HashiCorp Vault
- Encrypted secrets at rest

Pod Security

Restricted Policies

- Pod Security Standards (PSS)
- Minimal base images
- Non-root containers

Cost Controls on Kubernetes



Right-size Resources

Calibrate requests/limits accurately based on actual usage and avoid GPU idle time

Implement Auto-scaling

Use [node auto-scalers](#) + [cluster autoscaler](#) to match infrastructure to demand

Utilize Spot Instances

Prefer [spot/preemptible](#) node groups for Jobs with proper checkpointing

Track Costs Per Request

Monitor [\\$/req](#) in dashboards with proper FinOps tag labels

Service Mesh (Optional)



When to Consider a Service Mesh

Istio/Linkerd provide:

- Mutual TLS encryption (mTLS)
- Advanced retries & circuit breaking
- Configurable timeouts
- Precise canary weight control
- Detailed traffic telemetry

⊗ Adds latency and complexity—only use when you specifically need traffic policies or zero-trust networking.

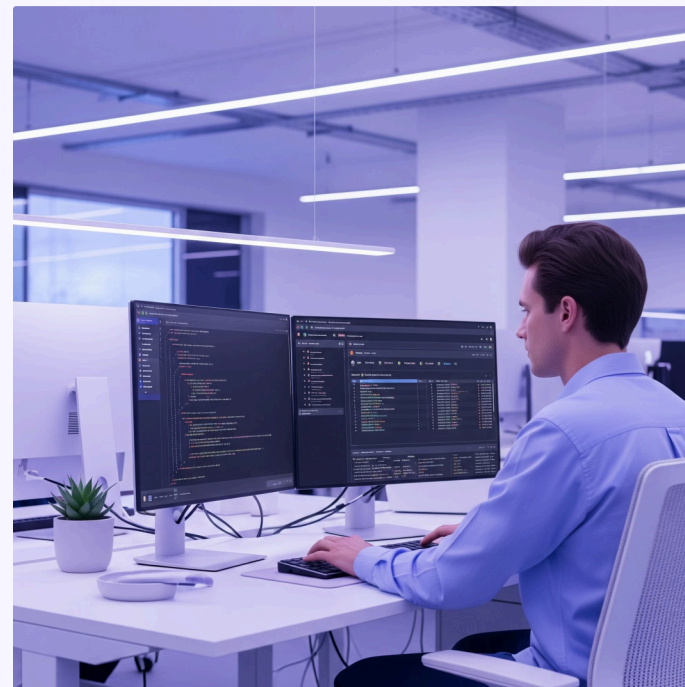
Troubleshooting Playbook

Essential Commands

```
kubectl get pods -A    # status overview  
kubectl describe pod   # events & scheduling  
kubectl logs          # app logs (-c for multi)  
kubectl top pod/node   # resource usage
```

Common Issues

- **imagePullBackOff**: Invalid image name or missing pull secrets
- **CrashLoopBackOff**: Application crashes on startup
- **OOMKilled**: Container exceeded memory limits
- **Unschedulable**: Not enough resources (especially GPUs)

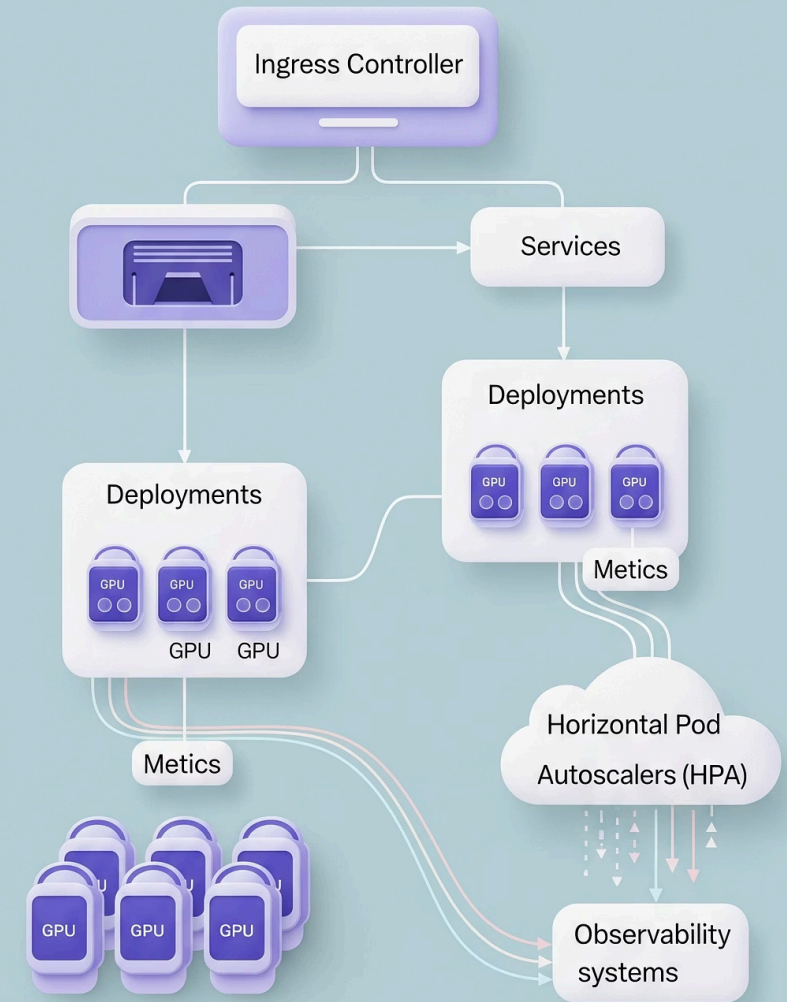


Debugging Tools

Install [k9s](#) for a terminal UI and [kube-ps1](#) to show your current context in your prompt.

Kubernetes ML Architecture

"Declarative manifests + autoscaling + observability = reliable AI ops."





Summary

K8s for AI

Kubernetes standardizes **deploy/scale/observe** workflows for AI services, providing a consistent platform across environments.

Core Components

- GPU scheduling
- Horizontal Pod Autoscaling
- Canary deployments
- Secure configurations

Supporting Systems

- CI/CD pipelines
- GitOps workflows
- Cost monitoring (FinOps)
- Comprehensive observability

Discussion Prompt

"For your Capstone, will you ship with simple rolling updates or implement canary + GitOps from day one? What's the trade-off?"

Simple Rolling Updates

Pros: Faster setup, less complexity, built-in

Cons: Limited control, all-or-nothing deployment

Canary + GitOps

Pros: Safer releases, better audit trail, infrastructure as code

Cons: More infrastructure, steeper learning curve

